INFO 4220 – Networks II: Market Design Spring 2025 – Assignment 1

Solution + Rubric

- to provide your code and output or your responses will not be considered valid

Part 1: Setup and tutorial (5 points)

For this part you will setup R in your computer and numerically compute a random assignment. If you don't have R currently installed, a simple way of quickly setting up a usable environment is installing R Studio. To do so follow the steps below:

1. If you don't have R installed in your computer, download and install R and R studio: https://posit.co/download/rstudio-desktop/#download
2. If you don't have java installed, also install java (you need Java JDK 11 or higher) : https://www.oracle.com/java/technologies/downloads/

After installing R, follow the steps below to numerically compute a random serial dictatorship

```
# Install and load the matchingMarkets library. If you want to look at
the documentation of the library for more info, look at:
https://cran.r-
project.org/web/packages/matchingMarkets/matchingMarkets.pdf

install.packages("matchingMarkets")
library(matchingMarkets)
# Create vectors with the preferences of Alice, Bob, Carol, and Denis
(Note: This is the example in the slides of L8: Probabilistic
Assignments. Object 1 is Apple, 2 is Orange, 3 is Pear, and 4 is
Strawberry)
alice<-c(1,2,3,4)
bob<-c(1,2,3,4)
carol<-c(2,1,4,3)
denis<-c(2,1,4,3)

# Combine the preferences into a matrix. Note that the way the
matchingMarkets library wants the matrix is different than what we saw
in class (the library wants people in columns and objects in rows)

matrix_prefs<-matrix(c(alice, bob, carol, denis), 4,4)
```

```
# Next, we create a vector with the four dictarors (we will use the
later for drawing random orders)

dictators <- c(1,2,3,4)

# Then we create an empty 4x4 matrix to store the results
results <- matrix(0, 4, 4)

# This vector will tell the rsd command that we have 1 of each object
nSlots<-c(1,1,1,1)

# The following vector contains the first preference (i.e. the most
preferred object) for each individual. What I am doing below is taking
the first row of the matrix with individuals' preferences and storing
it as vector (column) called first_prefs. I also create a variable
called count_first to count how many individuals got their most
preferred object. I will use these to keep track of the how many times
individuals are getting their most preferred object in my simulations

first_prefs <- matrix(c(matrix_prefs[1,]))
count_first <- 0

# Then repeat the same for second, third, and fourth preferences
second_prefs <- matrix(c(matrix_prefs[2,]))
third_prefs <- matrix(c(matrix_prefs[3,]))
fourth_prefs <- matrix(c(matrix_prefs[4,]))
count_second <- 0
count_third <- 0
count_fourth <- 0

# The loop below runs 10,000 random serial dictatorships and stores
the results in the matrix results we created above. It also counts how
many times individuals got their most preferred object, their second
most preferred object, etc.

for (i in 1:10000){
     # We first draw a random order of dictators
     order <- sample(dictators)

     # Then we run the serial dictatorship. This will return a matrix
with the results. The first column is the list of individuals, and the
second column is the objects they were assigned to
     assignment <- rsd(prefs = matrix_prefs, priority = order, nSlots
= nSlots)

     # The loop below updates the results matrix. It adds one to the
(object, person) cell if the person was assigned to the object
     for (i in 1:4){
          results[i, assignment[i,2]] <- results[i,
assignment[i,2]]+1
          }
```

```
      # The command below is going to compare the vector of first
preferences with the second column of the assignment matrix (which
contains the assignment). It will create a vector that contains a 1 if
person i was assigned to their most preferred object
      first_choice <- matrix(c(ifelse(first_prefs == assignment[,2], 1,
0)))


      # Then we just update the counter that counts how many times
individuals have been assigned their most preferred object
      count_first <- count_first + sum(first_choice)

      # We repeat this for the second, third, and fourth most preferred
objects
      second_choice <- matrix(c(ifelse(second_prefs == assignment[,2],
1, 0)))
      count_second <- count_second + sum(second_choice)
      third_choice <- matrix(c(ifelse(third_prefs == assignment[,2], 1,
0)))
      count_third <- count_third + sum(third_choice)
      fourth_choice <- matrix(c(ifelse(fourth_prefs == assignment[,2],
1, 0)))
      count_fourth <- count_fourth + sum(fourth_choice)
      }

# Finally we divide our results variable by 10,000 to express results
in terms of probabilities, and the counters by 10,000 to express them
as how many individuals on average get their most preferred object,
their second most preferred object, etc.

results <- results/10000
count_first <- count_first/10000
count_second <- count_second/10000
count_third <- count_third/10000
count_fourth <- count_fourth/10000
```

Now answer the following questions:

1. Compare your results matrix with the probabilistic assignment we computed in L8. Are the results similar? (1 point)

The resulting matrix (re-organizing it in the way it was presented in class is:

	Apple	Orange	Pear	Strawberry
Alice	.4156	0.0849	0.4146	0.0849
Bob	.4193	0.0783	0.4222	0.0802
Carol	0.818	0.4249	0.0854	0.4064
David	0.818	.4119	0.0778	0.4285

This is similar to what we obtained in class analytically. 5/12=0.4167 and 1/12=0.0833. It is not the same as this is a numerical simulation. The more iterations we do, the closest it should become.

Points assignment: 1 point for correct answer. Code was provided, so we are not evaluating/grading the code.

2. In your simulations, on average, how many people obtain their most preferred object, one of their 2 most preferred objects, or one of their 3 most preferred objects? (2 points)

On average: 1.6717 people got their most preferred object, 2 got one the two most preferred objects. 3.6717 people got one of their 3 most preferred objects.

Points assignment: 2 points for correct answer. 1.75 points if just listing how many people get the most preferred object, the second most preferred object, and the third most preferred object.

3. If instead of using a serial random dictatorship you used the probabilistic assignment produced by the eating algorithm and did a similar simulation. How many people obtain their most preferred object, one of their 2 most preferred objects, or one of their 3 most preferred objects? (2 points)

The eating algorithm results give us:

	Apple	Orange	Pear	Strawberry
Alice	1	0	0	0
Bob	0	0	1	0
Carol	0	1	0	0
David	0	0	0	1

This can be decomposed in the following 4 scenarios, each with probability 0.25:

	Apple	Orange	Pear	Strawberry
Alice	1	0	0	0
Bob	0	0	1	0
Carol	0	0	0	1
David	0	1	0	0

	Apple	Orange	Pear	Strawberry
Alice	1	0	0	0
Bob	0	0	1	0
Carol	0	1	0	0
David	0	0	0	1

	Apple	Orange	Pear	Strawberry
Alice	0	0	1	0
Bob	1	0	0	0
Carol	0	0	0	1
David	0	1	0	0

	Apple	Orange	Pear	Strawberry
Alice	0	0	1	0
Bob	1	0	0	0
Carol	0	1	0	0
David	0	0	0	1

If we run 10,000 simulations, each scenario would be chosen ~2,500 times.

In each of the scenarios two people get their most preferred object, and two their third most preferred object. That means that on average: 2 people got their most preferred object, 2 got one the two most preferred objects. 4 people got one of their 3 most preferred objects.

Points: 1 point for the probabilistic assignment and decomposition, 1 points for the reasoning of how many people get their most preferred object. Do not discount points if answer is listing how many people get the most preferred object, the second most preferred object, and the third most preferred object.

Part 2: Simulating a school assignment problem (25 points)

You have been hired by a school district in Central New York to decide which algorithm to use for their new school choice program. The school district and the community cannot decide

whether to use deferred acceptance or top trading cycle. For the next town hall meeting you have been tasked with producing a simulation that exemplifies the outcomes that would result from using each algorithm. To do so, follow the following assumptions/steps:

- There are 4 schools with capacities 10, 20, 15, 15
- There are 60 students that need to be assigned to schools
- For each school, create a random order of students (one different random order for each school)
- Simulate 5000 matchings. For each iteration:
    - Start by creating a random order of student preferences over school (for each iteration create new student preferences while keeping school priorities fixed)
    - Create the assignment that results from running DA. Count how many students get their first, second, or third preference.
    - Create the assignment that results from running TTC. Count how many students get their first, second, or third preference.
- At the end of the 5000 simulations, report how many students get their first, second, and third preference on average with each of the algorithms

Answer the following questions:

1. If the objective is to maximize the number of students that get their favorite schools, which algorithm would you recommend? (15 points)

When using TTC:

Students assigned to first preference: 50.391

Students assigned to second preference: 6.185

Students assigned to third preference: 2.181

With using DA:

Students assigned to first preference: 46.665

Students assigned to second preference: 10.776

Students assigned to third preference: 2.228

If we want to maximize the number of students that get assigned to their favorite school, we should use TTC.

Points assignment: 5 points for code, 5 points for TTC results, 5 points for DA result.

2. Is there any other reason why you would recommend using TTC? (5 points)

Using TTC is going to create assignments that are going to be efficient. We should care more about efficiency when schools don't have preferences over students (for example, when there are no schools that cater to kids with special talents or needs). Using an algorithm that is efficient guarantees that we cannot keep improving the matches by relocating kids across schools. Additionally, TTC is strategyproof for students. Thus, if schools don't have preferences (and thus no incentives to manipulate priority lists), and students have a dominant strategy, it is fairly straightforward how each agent should behave.

Point assignment: 2 points for saying it is efficient, 3 points for the explanation of when we care more about efficiency. Students do not need to say all the reasons I listed above. Providing 1 properly explained reason is enough for full credit.

3. Is there any other reason why you would recommend using DA? (5 points)

Using DA is going to create assignments that are stable. Typically, we should not care much about stability, as schools are required to accept all students, and they should not have preferences over students. However, as exemplified in the redesign of the NYC school matching system, often school do behave as they have preferences. In this case, we should prefer an algorithm that guarantees stability over efficiency.

Point assignment: 2 points for saying it is stability, 3 points for the explanation of when we care more about stability. Students do not need to say all the reasons I listed above. Providing 1 properly explained reason is enough for full credit.